

How Agent Dispatcher works, how it compares with Claude Code, and what to build next

Agent Dispatcher is a routing and context layer that runs **inside** Claude Code or Codex. It picks a specialist working method for each request, prepares a small ranked context packet from a local project index, and holds the final report to evidence that was actually collected. The host still runs the model and every tool.

THE PACK

27 roles

79 local skills · 31 external skills · 8 recipes · 19 MCP servers · 50 detection signals. Only the pieces a task selects are read.

LATEST EVIDENCE · 21 SEP

10/10 and 10/10

Dispatcher passed every trial in both reruns (stock: 9/10 and 10/10). The regression from 20 Sep, where it wrote protected cache files, is fixed.

THE OPEN QUESTION

+27–29% cost

On five small fixtures (1–14 files), Dispatcher matches stock quality at higher cost. It needs a large-repo suite to show what the index is worth.

IN THIS REPORT

02 · How a request flows

03 · Inside context preparation

04 · Roles, skills and layers

05 · Verification and honest reporting

06 · Compared with stock Claude Code

07 · Results across five runs

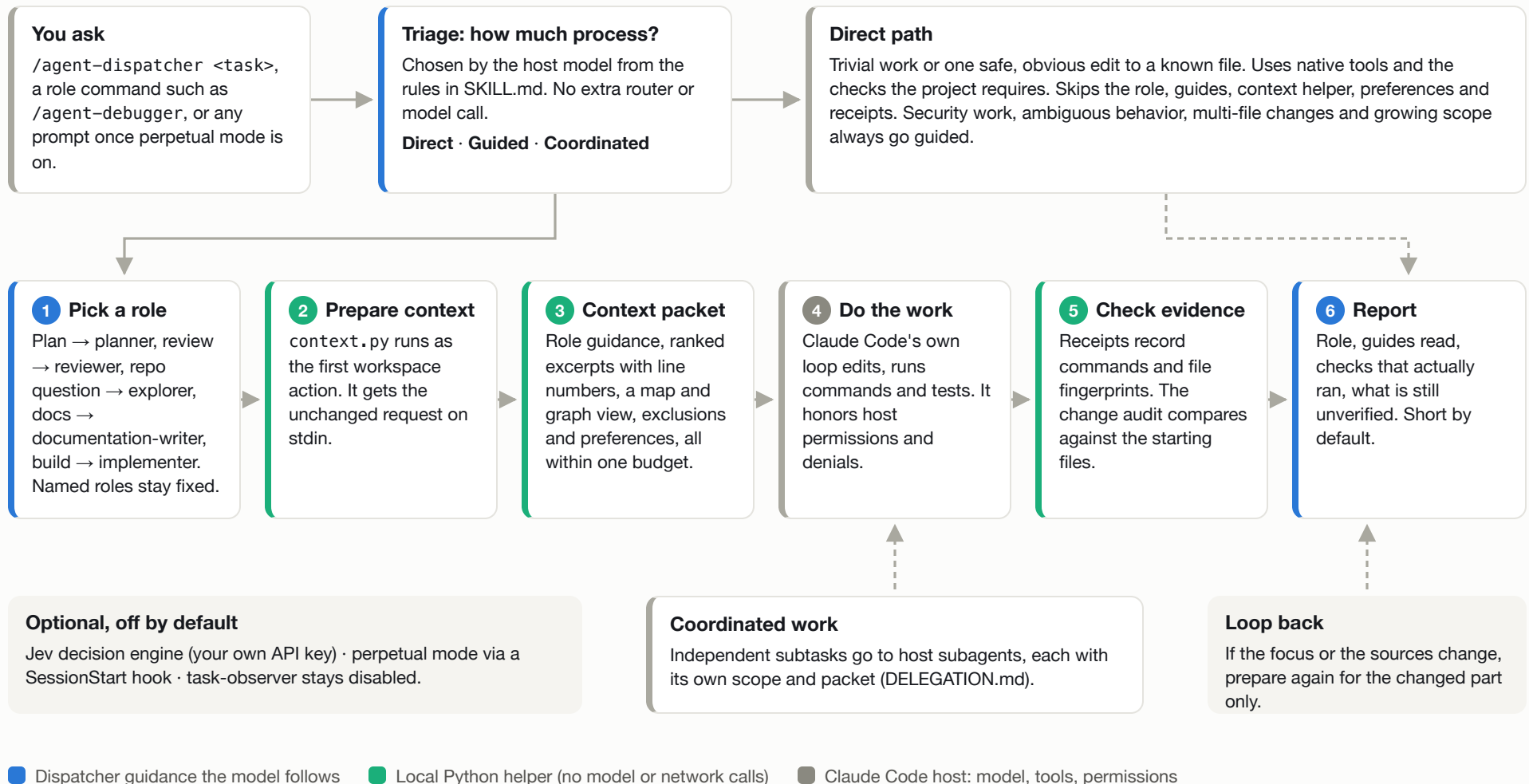
08 · Why small tasks can't show a win

09 · Recommended features and updates

10 · Roadmap, success test, sources

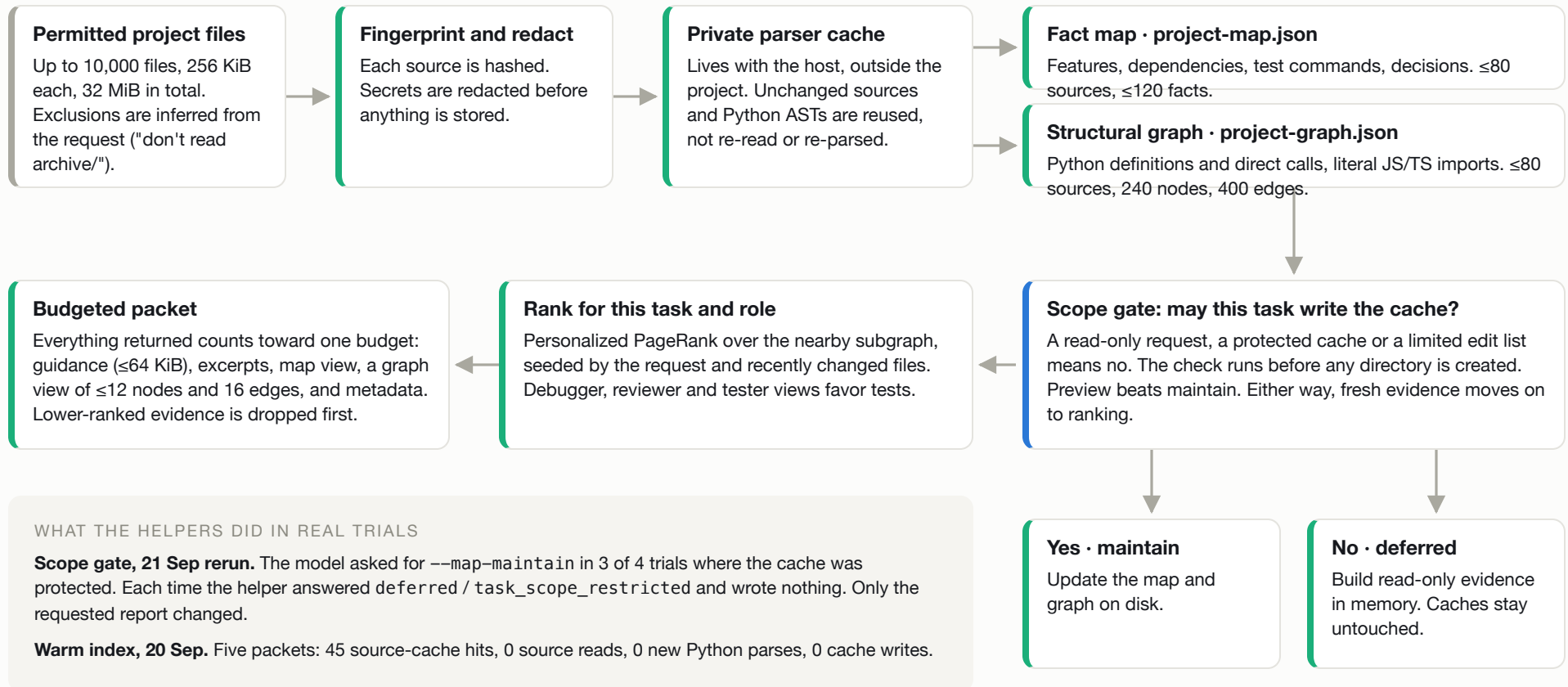
How a request flows through Dispatcher

Most of Dispatcher is instructions the host model follows, plus local Python helpers it calls. Claude Code still owns the model, the tools and the permission prompts.



Inside context preparation

One local command, `context.py`, turns the repository into a small, ranked, source-linked packet. It makes no model or network calls and runs no background service.



27 roles on a six-layer stack

A role is a working method, not a separate agent process. The host model adopts it, can chain roles within a turn, and loads skills in three steps so a large catalog stays cheap.

Agent
Who is responsible: mission, method, deliverable, boundaries

Recipe
How capabilities combine into one run

Skill
How to do one specialized thing

Reference
Detail that one step of a skill needs

MCP / tool
What external state can be reached

Verification
Evidence that the outcome works

Context engine: what each layer gets

Precedence: runtime policy › user and task instructions › repository rules › role contract › recipe › skill › references. A skill teaches but never authorizes. Permissions stay with the host.

CORE · 7

dispatchergeneralistimplementerplannerresearcherreviewer

tester

ENGINEERING · 13

ai-agent-engineerapi-integration-engineerarchitectdata-engineer

database-engineerdebuggerdevops-releaseexplorer

incident-responderperformance-engineerrefactoring-migration-specialist

security-auditorversion-control

KNOWLEDGE & BUSINESS · 5

automation-operationscontent-copywriterdata-analyst

documentation-writergrowth-marketing-strategist

PRODUCT & DESIGN · 2

product-managerui-ux-designer

PROGRESSIVE DISCLOSURE

1 · **Discovery.** INDEX.md and one-line descriptions only.

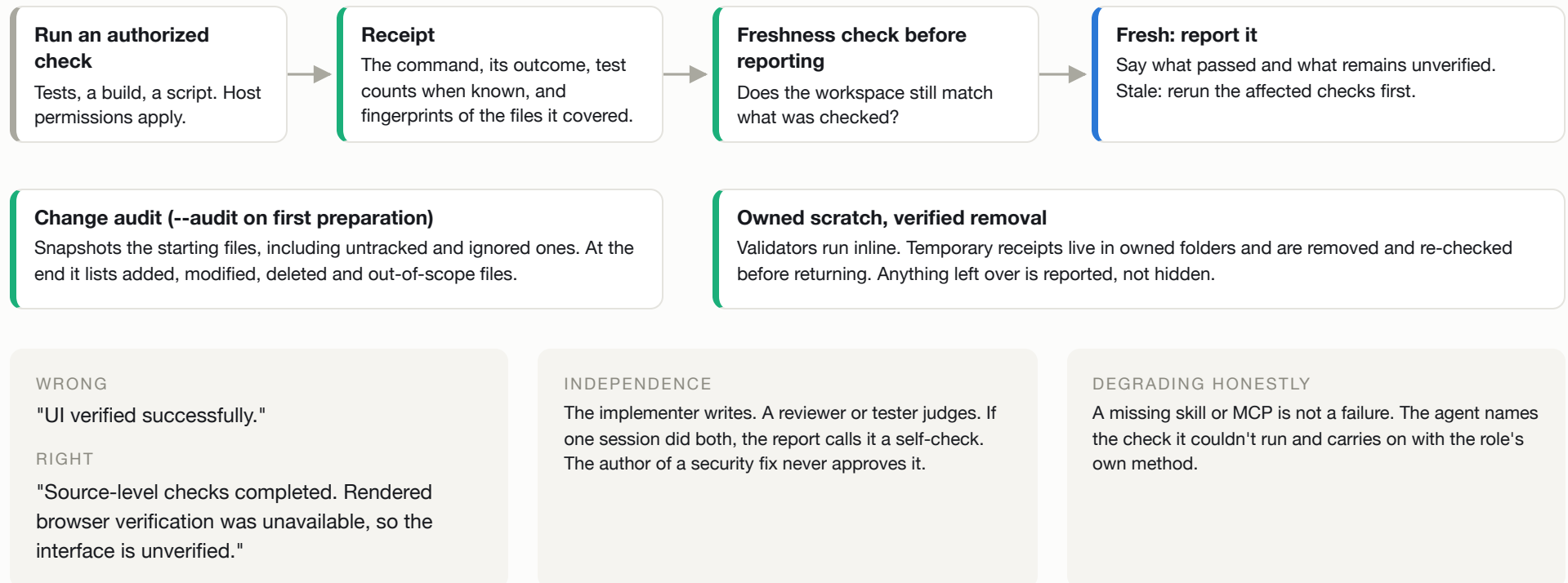
2 · **The skill.** Its SKILL.md is read once chosen, usually 1–5 per task.

3 · **References and scripts.** Read only when a step needs that piece.

Verification: report only what actually ran

The failure this pack guards against most is a confident report of a check that never ran. Each word below is a separate claim that needs its own evidence.

created ≠ executed ≠ tested ≠ reviewed ≠ deployed ≠ verified



Compared with stock Claude Code

Dispatcher isn't a separate agent competing with Claude Code. It's an overlay: the same host, plus a method, a context packet and evidence rules. The fair test is Claude Code alone against Claude Code with Dispatcher.

	STOCK CLAUDE CODE	+ AGENT DISPATCHER
Model, tools, permissions	Native	Same host, unchanged. Dispatcher never switches the model or effort silently.
Working method	One general-purpose approach	27 role methods. Triage into direct, guided or coordinated work.
Skills	Native skill discovery	Curated catalog of 79 local and 31 external skills, role loadouts, three-step loading.
Finding the right code	Search and read on demand	Ranked, budgeted packet with line-level excerpts, a fact map and a call/import graph.
Repeat work	Rediscovered each session; CLAUDE.md memory	Fingerprinted indexes and a private parser cache reuse unchanged sources.
Subagents	Native	Used only for independent subtasks, each with a bounded scope.
Protecting files	Permission modes and prompts	Adds scope-gated cache writes and a change audit against the starting files.
Final report	The model's own judgment	Receipts with freshness checks. Unrun checks are named as unverified.
Setup health	/doctor for the host	Adds a doctor and an inventory of skills, tools and MCP servers.

WHAT IT COSTS TODAY

+27–29%

provider-estimated cost in the 21 Sep reruns

+10.5–11.9 s

median extra time per matched task pair

+1.0–1.3K

median extra output tokens per matched pair

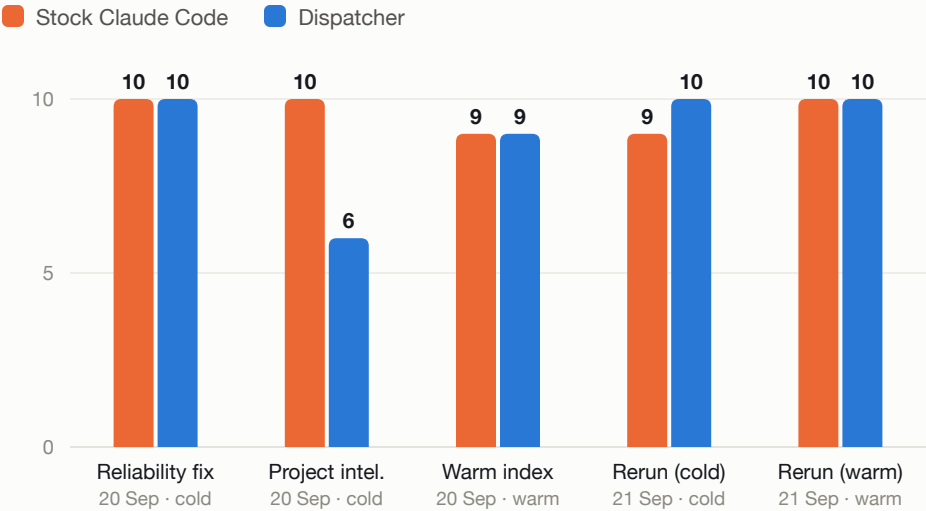
WHAT IT BUYS TODAY

Equal or better acceptance on every task in both reruns, plus no leftover scratch files. Stock's one failure left two helper scripts outside the project.

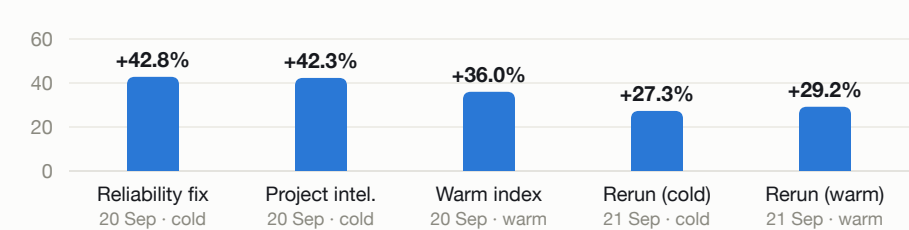
Results across five matched runs

Same five fixtures, Claude Opus 5 at high effort, Claude Code 2.1.150, seed 20260919, 10 trials per condition. Only the Dispatcher package version changed between runs.

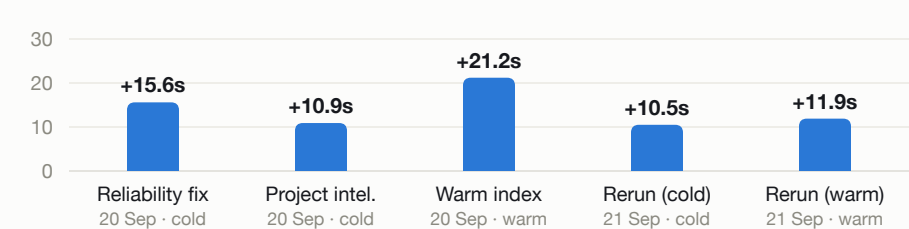
Accepted trials, out of 10



Dispatcher's extra cost vs stock, % of total



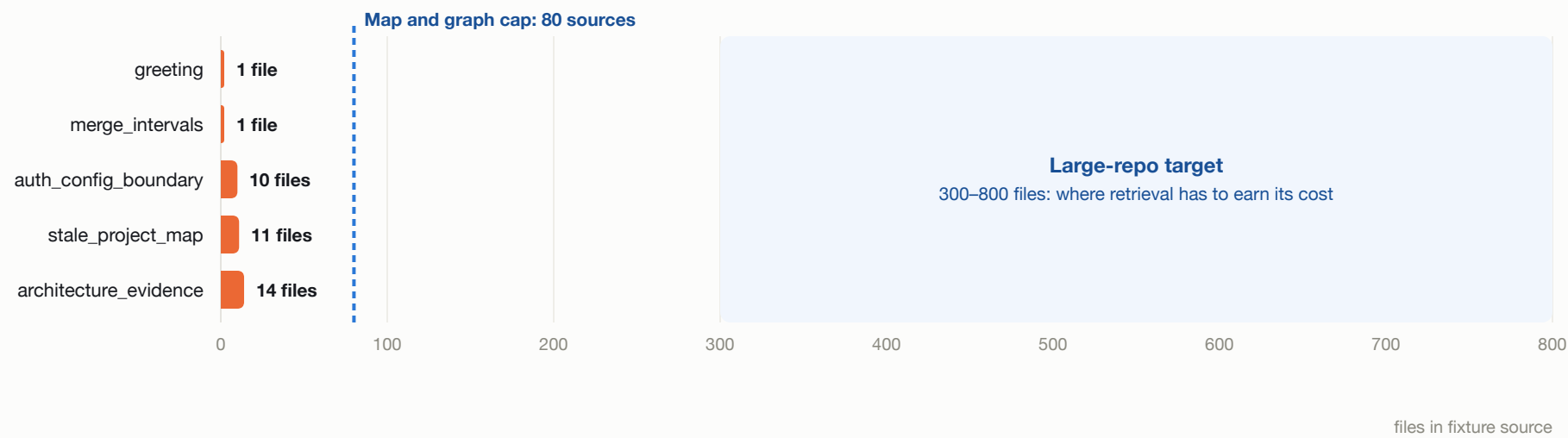
Extra time per matched pair, median seconds



RUN	STOCK OK	DISP. OK	STOCK \$	DISP. \$	STOCK \$/ACCEPTED	DISP. \$/ACCEPTED	STOCK MEDIAN S	DISP. MEDIAN S
Reliability fix · 20 Sep · cold	10/10	10/10	\$3.61	\$5.16	\$0.361	\$0.516	48.7	61.6
Project intel. · 20 Sep · cold	10/10	6/10	\$3.47	\$4.94	\$0.347	\$0.823	49.5	52.8
Warm index · 20 Sep · warm	9/10	9/10	\$3.75	\$5.10	\$0.417	\$0.567	57.4	77.1
Rerun (cold) · 21 Sep · cold	9/10	10/10	\$3.75	\$4.77	\$0.416	\$0.477	50.1	63.3
Rerun (warm) · 21 Sep · warm	10/10	10/10	\$3.67	\$4.75	\$0.367	\$0.475	49.8	63.9

The small suite has no room for Dispatcher to win

Stock Claude reads a 14-file project in a single pass, so an index, a graph and a ranked packet can only add cost there. A large repository is where retrieval has to prove itself, and the current index caps stop well short of that.



FINAL ANSWERS AREN'T THE OVERHEAD

1,107 vs 1,454

median characters in the final answer, Dispatcher vs stock (cold rerun). Dispatcher's reports are shorter, so the extra output tokens come from the working phase.

GUIDANCE THE MODEL READS

28.7K / 48.5K

characters of role and guide text returned across 10 Dispatcher trials (cold / warm). These are partial lower bounds, not token counts.

THE PROMPT IS TYPED TWICE

~1.0-1.2K chars

per context.py call. The model re-sends the full request on stdin as output tokens. The helper accepts up to 16,000 characters.

What to add or update next

In priority order. Items 1–3 are cheap and make the large-repo run measurable. Items 6–8 wait until that run shows where the cost actually is.

BEFORE THE LARGE-REPO RUN

1 Scale the index caps to the repo UPDATE

The map and the graph each index at most 80 sources, and the graph at most 240 nodes and 400 edges. On a 500-file repo, 84% of files get no structural coverage. Make the caps grow with repo size, or choose the 80 by task relevance. Report omitted counts in every eval.

2 Let the helper choose preview or maintain UPDATE

The model picked `--map-maintain` in 3 of 4 protected trials, and only enforcement saved the result. Infer the mode inside `context.py` from the same scope signals, and delete the flag rule from `SKILL.md`. That means less instruction text and one fewer way to fail.

3 Close the eval blind spots UPDATE

Map use shows as 'unknown' in 4 of 4 eligible trials in both reruns, because maintenance runs through `context.py`. The chosen role is unknown in all 20 trials. Attribute both from the context payload before measuring a feature you're counting on.

WITH THE LARGE-REPO RUN

4 The large-repo fixture set ADD

300–800 files. Answers should depend on a caller or dependency the prompt never names. Include one stale doc and one multi-file change. Time first-use indexing separately from warm runs. Pilot until stock fails at least one task, then freeze. Run Codex as well as Claude.

5 Automated claim checking ADD

Claim accuracy has been 0/0 observed in every run, and 24 human reviews from 19 Sep are still pending. Check final-answer claims such as 'tests pass', 'nothing else changed' and test counts against executed commands, receipts and the change audit. Honest reporting is Dispatcher's main quality promise, and nothing measures it yet.

LATER, ONLY IF THE DATA SUPPORTS IT

6 Pass the prompt without re-typing it ADD

A Claude Code `UserPromptSubmit` hook could save the request for `context.py` to read. It's small today, but the cost grows with long prompts.

7 Wider structural extraction UPDATE

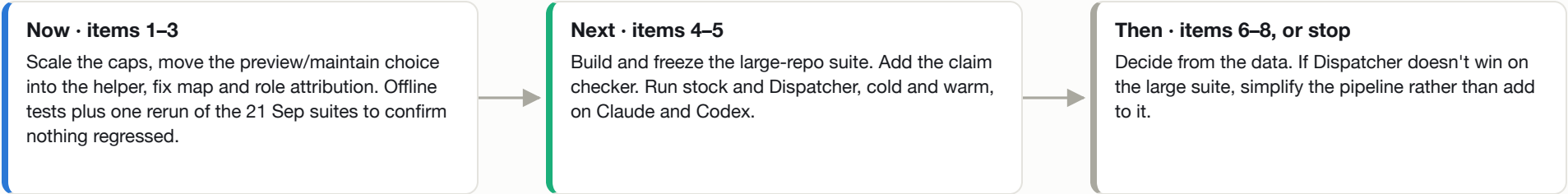
The graph covers Python definitions and calls, plus literal JS/TS imports. Pick the fixture's language deliberately. Extend extraction only if TypeScript or Go graphs turn out to matter, and keep the no-dependency rule.

8 Find where the extra 1K output tokens go MEASURE

Profile guidance reads and reasoning on the big fixture before trimming instructions. Cutting blind risks losing the rules that fixed the regression.

Roadmap and the test for success

Judge each change by **cost per accepted task**, counting every attempt, subagent and verification step. Adopt a change when quality goes up, or stays equal at lower cost.



COST PER ACCEPTED TASK, 21 SEP

	STOCK	DISPATCHER
Cold rerun	\$0.416	\$0.477
Warm rerun	\$0.367	\$0.475

The large-repo suite should push the Dispatcher column below stock, or show acceptance that stock can't reach.

SOURCES

Implementation: skills/agent-dispatcher/SKILL.md, CONTEXT.md, VERIFICATION.md, DELEGATION.md; context.py, project_map.py, project_graph.py, parser_cache.py, verification.py, change_audit.py; docs/architecture.md, docs/project-intelligence.md, docs/verification.md; templates/.

Evaluations (dist/evals): opus5-reliability-fix-20260920 (pilot 20260920T053330Z-46a6325f) · opus5-project-intelligence-20260920 (20260920T175109Z-3aa2a9cc) · opus5-incremental-warm-20260920 (20260920T195626Z-f5b1244e; SUMMARY.md corrected acceptance) · opus5-rerun-cold-20260921 (20260921T041303Z-09b9f144) · opus5-rerun-warm-20260921 (20260921T043300Z-24d9846a).

Rerun setup: HEAD 2025727, fixture digests identical to the 20 Sep runs (e830a604..., ffc194eb...), same seed, model, effort and CLI. Smoke 8/8 passed. Total estimated usage \$18.80.

Limits: five small fixtures and two repetitions. Artifact and scope checks are automated; there are no human ratings. Costs are provider estimates. Codex has not been evaluated live.